

UNIDAD 2 · SEMANA 2

Llamadas al sistema y E/S de bajo nivel

📅 Semana 2 · fechas por confirmar </> Laboratorio TD1 (resuelto + desafío IA)

~ 4 animaciones

LABORATORIO TD1 · RESUELTO

Leer y escribir con `read` y `write`

Los fundamentos de la programación de sistemas: cómo entra un programa al núcleo, cómo se escribe y se lee en binario, y por qué hay que comprobar cada resultado.

✓ Informe del laboratorio

Entrega este laboratorio con un informe corto en LaTeX, con la plantilla del curso: cada ejercicio explicado, el código que importa y lo que ejecutaste en la terminal. Puedes usar IA generativa para escribir el LaTeX: decláralo y verifica cada resultado. [Plantilla, ejemplo y guía de Prism](#) →

EJERCICIOS

- | | | | |
|---|---|---|--|
| 1 | Cómo se implementa una llamada al sistema | 4 | ¿ <code>perror</code> es una llamada al sistema? |
| 2 | <code>echo</code> con <code>write</code> | 5 | Copiar con <code>read</code> |
| 3 | Escribir un entero en binario | 6 | Leer el entero |
| | | 7 | Estructuras en binario y relleno |

TD1 · 1

● Resuelto · salida real

Cómo se implementa una llamada al sistema

Enunciado. Una llamada al sistema necesita una instrucción especial del procesador (`syscall` en x86-64, `svc` en ARM, `ecall` en RISC-V). Desensambla la biblioteca de C con `objdump -d` y encuentra cómo le dice el programa al núcleo *qué* llamada quiere. Luego observa con `strace` las llamadas que hace tu `echo`.

Solución



Salida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A4 -E '^[0-9a-f]+ <getppid@>'
0000000000ff710 <getppid@GLIBC_2.2.5>:
    ff710:    endbr64
    ff714:    mov     $0x6e,%eax
    ff719:    syscall
    ff71b:    ret
```

Salida real

```
$ objdump -d --no-show-raw-insn /usr/lib/x86_64-linux-gnu/libc.so.6 | grep -
A6 -E '^[0-9a-f]+ <__write_nocancel@>'
000000000129060 <__write_nocancel@GLIBC_PRIVATE>:
    129060:    endbr64
    129064:    mov     $0x1,%eax
    129069:    syscall
    12906b:    cmp     $0xffffffffffffffff,%rax
    129071:    ja      129078 <__write_nocancel@GLIBC_PRIVATE+0x18>
    129073:    ret
```

El número de la llamada va en el registro `eax` (110 = `getppid`, 1 = `write` en x86-64) y los argumentos en `rdi`, `rsi`, `rdx` ... La instrucción `syscall` cambia la CPU a modo núcleo y salta a la entrada del núcleo, que usa `eax` como índice en su tabla de llamadas. Al volver, el resultado está en `rax`: un valor entre -4095 y -1 significa error, y la envoltura de glibc lo convierte en `-1 + errno` (eso hace el `cmp $0xffffffffffffffff,%rax`).

Salida real

```
$ strace -e trace=write ./echo hola mundo > /dev/null
write(1, "hola", 4)                = 4
write(1, " ", 1)                  = 1
write(1, "mundo", 5)              = 5
write(1, "\n", 1)                 = 1
+++ exited with 0 +++
```

Cuatro llamadas `write`: una por palabra, una por el espacio y una por el salto de línea. Cada una es un viaje de ida y vuelta al núcleo; en la unidad 4 verás cómo la biblioteca estándar agrupa escrituras en un búfer para ahorrarlas.

TD1 · 2

● Resuelto · probado

echo con write(2)

Enunciado. Escribe un programa `echo` que copie sus argumentos en la salida estándar usando solo `write(2)`. Debe comprobar los errores de `write` con `exit_if`. Pruébalo con `./echo blabla < . 1>&0`.

Solución



C tds/td1/echo.c

```

/*
 * TD1 - exercise 3: echo with write(2).
 * Copies its arguments to standard output, separated by spaces, followed by
 a newline.
 * Try the error path with: ./echo blabla < . 1>&0 (stdout becomes a
 directory: write fails)
 */
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    for (int i = 1; i < argc; i++) {
        size_t len = strlen(argv[i]);
        ssize_t rc = write(STDOUT_FILENO, argv[i], len);
        exit_if(rc == -1, "write");
        /* write may write less than asked (pipe or socket full): it is not
 an error,
            but this program must then say so. write_all() in libutils loops
 instead. */
        exit_if((size_t)rc != len, "write: partial write");

        if (i < argc - 1)
            exit_if(write(STDOUT_FILENO, " ", 1) == -1, "write space");
    }
    exit_if(write(STDOUT_FILENO, "\n", 1) == -1, "write newline");
    return EXIT_SUCCESS;
}

```

Salida real

```

$ ./echo hola mundo
hola mundo

```

Salida real

```
$ ./echo blabla < . 1>&0
write: Bad file descriptor
[exit status 1]
```

`< .` abre el directorio actual como entrada estándar (descriptor 0) y `1>&0` hace que la salida (1) sea una copia del descriptor 0. Un directorio abierto en solo lectura no admite escrituras: `write` devuelve `-1` con `errno = EBADF`, y `exit_if` lo informa.

TD1 · 3

● Resuelto · probado

Escribir un entero en binario

Enunciado. Escribe `write_number`: escribe en la salida estándar el contenido en memoria de un `long` que vale 5 y, después, el número de bytes que escribió la primera llamada. ¿Qué ves en la terminal? Mira la salida con `od`.

Solución



C tds/td1/write_number.c

```
/*
 * TD1 - exercise 4: write the memory content of a long, then the number of
 * bytes written.
 * The output is binary, not text: look at it with ./write_number | od -A d
 * -t x1
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    long value = 5;
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);    /* 8 bytes on
x86-64 */
    exit_if(written == -1, "write value");

    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./write_number | od -A d -t x1
00000000 05 00 00 00 00 00 00 00 08 00 00 00 00 00 00
00000016
```

En la terminal no se ve nada legible: los bytes `05 00 00 00 00 00 00 00` no son caracteres imprimibles. `od` muestra la verdad: 8 bytes del `long` en orden *little-endian* (el byte menos significativo primero, como guarda los enteros un procesador x86-64) y 8 bytes con el valor 8, que es lo que devolvió el primer `write`. `write` no convierte nada a texto: copia bytes.

TD1 · 4

● Resuelto

¿`perror` es una llamada al sistema?

Respuesta

No. `perror` está en la sección 3 del manual: es una función de la biblioteca de C que lee `errno`, busca el texto del error y lo escribe en la salida de error... usando, por dentro, la llamada al sistema `write`. Puedes comprobarlo con

```
strace ./echo blabla < . 1>&0: aparece un
write(2, "write: Bad file descriptor\n", ...).
```

TD1 · 5

● Resuelto · probado

Copiar la entrada en la salida con `read(2)`

Enunciado. Escribe `read`: lee la entrada estándar con `read` en un arreglo de tamaño `BUFFER_SIZE` y la escribe en la salida con `write`, hasta que la entrada se cierre (`Ctrl + D`). Pruébalo como `cat`, copiando un archivo y con un directorio como entrada.

Solución

C tds/td1/read.c

```

/*
 * TD1 - exercise 6: copy standard input to standard output with
 * read(2)/write(2).
 * Stops at end of file (Ctrl-D in a terminal). Usages:
 * ./read          ./read < file          ./read < in > out
 * ./read < . (error)
 */
#include <errno.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#ifndef BUFFER_SIZE
#define BUFFER_SIZE 4096
#endif

int main(void)
{
    char buffer[BUFFER_SIZE];

    for (;;) {
        ssize_t n = read(STDIN_FILENO, buffer, BUFFER_SIZE);
        if (n == -1 && errno == EINTR)
            continue;
        exit_if(n == -1, "read");
        if (n == 0) /* end of file: the writer closed its
side */
            return EXIT_SUCCESS;

        /* write can be partial: loop until the n bytes are out (see
write_all in libutils). */
        ssize_t done = 0;
        while (done < n) {
            ssize_t rc = write(STDOUT_FILENO, buffer + done, (size_t)(n -
done));
            exit_if(rc == -1, "write");
            done += rc;
        }
    }
}

```

Salida real

```
$ ./read < input.txt
first line
second line
```

Salida real

```
$ ./read < binary.bin > copy.bin && cmp binary.bin copy.bin && echo identical
identical
```

Salida real

```
$ ./read < .
read: Is a directory
[exit status 1]
```



Dos detalles que separan un programa correcto de uno que "casi siempre" funciona

- `read` devuelve **0** al final del archivo, **-1** en error y, si no, *hasta* `BUFFER_SIZE` bytes: puede devolver menos (una tubería, una terminal o una señal cortan la lectura).
- `write` también puede escribir menos de lo pedido: por eso el bucle interno repite hasta sacar los `n` bytes. Si se interrumpe por una señal antes de escribir nada, falla con `EINTR` y basta con reintentar.

TD1 · 6

● Resuelto · probado

Leer el entero de `write_number`

Enunciado. Escribe un programa que lea lo que produce `write_number` y lo muestre con `printf`.

Solución



C tds/td1/read_number.c

```

/*
 * TD1 - exercise 7: read what write_number produces and print it as text.
 * ./write_number | ./read_number      →  value = 5, bytes written = 8
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

/* read() may return fewer bytes than asked (e.g. from a pipe): loop until
`len` bytes. */
static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    long value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("value = %ld, bytes written = %zd\n", value, written);
    return EXIT_SUCCESS;
}

```

Salida real

```

$ ./write_number | ./read_number
value = 5, bytes written = 8

```

`read_exact` repite `read` hasta obtener exactamente los bytes pedidos: a través de una tubería los 16 bytes podrían llegar en dos trozos. Funciona porque los dos programas corren en la misma máquina: comparten el tamaño de `long` y el orden de los bytes. Entre máquinas distintas habría que acordar un formato (por ejemplo, orden de red con `htonl`, unidad 8).

TD1 · 7

● Resuelto · probado

Estructuras en binario y relleno

Enunciado. Convierte los programas anteriores en `write_struct` y `read_struct` para una `struct nopad { char c1; long l; char c2; }` declarada en `utils.h`, con `'a'`, 5 y `'b'`. ¿Cuánto mide la estructura? ¿Cambia con `-03`? ¿Cómo se puede reducir?

Solución

```
C tds/td1/write_struct.c

/*
 * TD1 - exercise 8: write a structure in binary. struct nopad is declared in
 * utils.h
 * so that write_struct and read_struct agree on its layout.
 */
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(void)
{
    struct nopad value = { .c1 = 'a', .l = 5, .c2 = 'b' };
    ssize_t written;

    written = write(STDOUT_FILENO, &value, sizeof value);
    exit_if(written == -1, "write value");
    exit_if(write(STDOUT_FILENO, &written, sizeof written) == -1, "write
count");
    return EXIT_SUCCESS;
}
```

C tds/td1/read_struct.c

```

/*
 * TD1 - exercise 8: read the structure written by write_struct.
 * ./write_struct | ./read_struct      → { c1 = 'a', l = 5, c2 = 'b' },
bytes written = 24
 */
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

static void read_exact(int fd, void *buf, size_t len, const char *what)
{
    char *p = buf;
    size_t got = 0;

    while (got < len) {
        ssize_t n = read(fd, p + got, len - got);
        exit_if(n == -1, what);
        exit_if(n == 0, "unexpected end of file");
        got += (size_t)n;
    }
}

int main(void)
{
    struct nopad value;
    ssize_t written;

    read_exact(STDIN_FILENO, &value, sizeof value, "read value");
    read_exact(STDIN_FILENO, &written, sizeof written, "read count");
    printf("{ c1 = '%c', l = %ld, c2 = '%c' }, bytes written = %zd\n",
           value.c1, value.l, value.c2, written);
    return EXIT_SUCCESS;
}

```

Salida real

```

$ ./write_struct | ./read_struct
{ c1 = 'a', l = 5, c2 = 'b' }, bytes written = 24

```

C tds/td1/struct_sizes.c

```

/*
 * TD1 - exercise 8 (answers): size and padding of struct nopad, and two ways
 to shrink it.
 * The compiler aligns `long` on 8 bytes: 7 padding bytes after c1 and 7
 after c2 (arrays of
 * the structure must keep every `l` aligned). -O3 does not change the
 layout: it is part of
 * the ABI, not an optimisation.
 */
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

struct reordered {          /* biggest member first: only 6 padding bytes at
 the end */
    long l;
    char c1;
    char c2;
};

struct __attribute__((packed)) packed { /* no padding at all (slower,
unaligned access) */
    char c1;
    long l;
    char c2;
};

int main(void)
{
    printf("struct nopad      : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct nopad),
        offsetof(struct nopad, c1), offsetof(struct nopad, l),
        offsetof(struct nopad, c2));
    printf("struct reordered : size %2zu  offsets l=%zu c1=%zu c2=%zu\n",
sizeof(struct reordered),
        offsetof(struct reordered, l), offsetof(struct reordered, c1),
        offsetof(struct reordered, c2));
    printf("struct packed    : size %2zu  offsets c1=%zu l=%zu c2=%zu\n",
sizeof(struct packed),
        offsetof(struct packed, c1), offsetof(struct packed, l),
        offsetof(struct packed, c2));
}

```

```
return EXIT_SUCCESS;
}
```

Salida real

```
$ ./struct_sizes
struct nopad      : size 24  offsets c1=0 l=8 c2=16
struct reordered  : size 16  offsets l=0 c1=8 c2=9
struct packed     : size 10  offsets c1=0 l=1 c2=9
```

La estructura mide **24 bytes** aunque sus campos suman 10: el compilador alinea `long` a 8 bytes y agrega 7 bytes de relleno después de `c1` y 7 al final (para que, en un arreglo de estructuras, cada `l` siga alineado). `-O3` no cambia nada: la disposición en memoria es parte de la *ABI*, no una optimización. Para reducirla: ordenar los campos de mayor a menor (16 bytes) o empaquetarla con `__attribute__((packed))` (10 bytes, con accesos desalineados más lentos y no portables).

⚠ Corregido respecto a la versión heredada

`read_struct.c` no compilaba: la estructura no estaba en `utils.h` y usaba un campo `c` que no existe.

Lista de cotejo

Desafío con IA generativa · Un `cat` que mide cuántas llamadas al sistema hace



Objetivo. Extender `read.c` para que reciba el tamaño del búfer como argumento, medir con `strace -c` y con el tiempo de ejecución cuántas llamadas `read` y `write` hace según el tamaño, y explicar la curva.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Tengo este programa en C que copia la entrada estándar a la salida con `read/write` y un búfer de tamaño fijo [pega `read.c`]. Ayúdame a: (1) recibir el tamaño del búfer por argumento (1 byte a 1 MiB) usando `malloc`; (2) escribir un script bash que, para tamaños 1, 16, 256, 4096, 65536 y 1048576, copie un archivo de 50 MB, mida el tiempo con `/usr/bin/time -f %e` y cuente las llamadas con `strace -c`; (3) explicarme por qué el tiempo baja tanto al principio y luego se estabiliza. No me des la conclusión antes de que yo vea los números.

Cómo verificar el resultado

- El archivo copiado es idéntico (`cmp`) para todos los tamaños.
- El número de `read` es $\approx$ tamaño del archivo / tamaño del búfer (más una lectura final que devuelve 0).
- Puedes explicar con tus palabras el costo fijo de cada llamada al sistema (cambio de modo).

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.